

JSONPlaceholder を使った検索・更新・削除アプリケーション課題

目次

- JSONPlaceholder を使った検索・更新・削除アプリケーション課題
 - 目次
 - 1. 課題概要
 - 2. 技術的前提・ルール
 - 3. 使用 API
 - 4. 画面構成
 - 5. 一覧画面
 - 5.1 役割
 - 5.2 検索フォーム仕様
 - 5.3 バリデーション仕様（一覧画面）
 - 5.4 検索の挙動
 - 5.5 一覧表示
 - 5.6 ページネーション
 - 5.7 エラー処理（一覧画面）
 - 6. 参考 : *_like を使った部分一致サンプル
 - 6.1 概要
 - 6.2 title を部分一致検索するサンプル関数（参考）
 - 7. 詳細画面
 - 7.1 役割
 - 7.2 初期表示
 - 7.3 バリデーション（詳細画面・更新時）
 - 7.4 更新処理（PUT/PATCH）
 - 7.5 削除処理（DELETE）

1. 課題概要

素の JavaScript と JSONPlaceholder を用いて以下を実装する課題です。

- 一覧検索（条件検索 + 部分一致）
 - ページネーション（10 件ごと）
 - 詳細表示
 - 更新
 - 削除
 - バリデーション
 - `fetch` と `try ~ catch` によるエラーハンドリング
-

2. 技術的前提・ルール

- 使用技術
 - HTML
 - CSS
 - **JavaScript（素の JS のみ）**
- 使用禁止
 - jQuery, React などのフレームワーク／ライブラリ
 - axios など **fetch** の代替ライブラリ
 - バリデーションやページネーションを補助するライブラリ
- HTTP 通信
 - すべて **fetch** を使用すること
 - すべての API 呼び出しは **try { ... } catch (error) { ... }** の中で実行すること
- 成功・失敗の UI 表示
 - API 成功時・失敗時の両方で、ユーザーに状態がわかるようにすること
 - 例：画面上のメッセージ領域、アラート等
 - メッセージの表示方法・デザインは実装者に任せる
- UI 全般
 - 見た目のデザインは自由
 - バリデーションエラーの表示位置も自由（フィールド付近／フォーム上部など）

3. 使用 API

- ベース URL
<https://jsonplaceholder.typicode.com>
- 主に使用するエンドポイント
 - 一覧取得 : **GET /posts**
 - 詳細取得 : **GET /posts/:id**
 - 更新 : **PUT /posts/:id** または **PATCH /posts/:id**
 - 削除 : **DELETE /posts/:id**

JSONPlaceholder の注意（重要）

JSONPlaceholder はテスト用の **ダミー API** です。

POST / PUT / PATCH / DELETE の呼び出しは **成功したようなレスポンス** を返しますが、**サーバー上のデータは実際には変更されません**。

そのため、更新・削除後に再度 **GET /posts** や **GET /posts/:id** を呼ぶと、元のデータが返ってきます。

本課題では、「HTTP メソッドとエラーハンドリングの練習」が目的であり、**永続的な更新・削除は行われないことを理解してください**。

4. 画面構成

- 一覧画面
- 詳細画面

5. 一覧画面

5.1 役割

- `/posts` のデータを対象とした一覧表示画面
- 検索フォームから条件検索
- 検索結果のページネーション（10 件ごと）
- 各行クリックで詳細画面へ遷移

5.2 検索フォーム仕様

フォームに用意する項目：

1. ユーザー (userId)

- `select` 要素
- value: `1~10`
- 表示ラベル：
 - value=1 → 「ユーザー 1」
 - ...
 - value=10 → 「ユーザー 1 0」
- 「未選択（全ユーザー）」状態があっても良い

2. 投稿 ID (id)

- `input` 要素

3. タイトル (title)

- `input` 要素

4. 本文 (body)

- `textarea` 要素

画面上のラベル表記

`userId` / `id` / `title` / `body` といった英語のプロパティ名はそのまま表示せず、「ユーザー」「投稿 ID」「タイトル」「本文」など、ユーザーにとって分かりやすい文言を使用してください。

5.3 バリデーション仕様（一覧画面）

- 実行タイミング：検索ボタン（submit）押下時
 - ルール
 - **id**
 - 入力されている場合のみチェック
 - 数値に変換できるか（整数であること）
 - **title**
 - 入力されている場合のみチェック
 - 文字数が **10 文字以内**
 - **body**
 - 入力されている場合のみチェック
 - 文字数が **50 文字以内**
 - エラー時
 - API は呼び出さず、エラーメッセージを表示
 - エラー表示位置とスタイルは実装者に任せる
-

5.4 検索の挙動

- 全項目未入力／未選択で検索した場合
 - **GET /posts** を呼び出し、全件取得する
- いずれかの項目が入力／選択されている場合
 - 条件の解釈
 - **userId** : 完全一致（数値比較）
 - **id** : 完全一致（数値比較）
 - **title** : 部分一致（後述）
 - **body** : 部分一致（後述）

部分一致の仕様は JSONPlaceholder の仕様に依存するので [6. 参考 : *_like を使った部分一致サンプル](#) で別途説明します。

5.5 一覧表示

- 少なくとも以下を表示すること
 - ユーザー（**userId** を「ユーザー 3」などと変換した文字列）
 - タイトル
 - 各行またはタイトル部分をクリックすると、詳細画面へ遷移
 - 例：`detail.html?id=3` のようにクエリパラメータで **id** を渡す
-

5.6 ページネーション

- 1ページあたり **10 件** 表示
 - ページを切り替える UI
 - 「前」 ボタン
 - 「次」 ボタン
 - 挙動
 - 検索結果の配列をもとに、クライアント側でページ分割する
 - 最初のページでは「前」 ボタンを無効／非表示
 - 最後のページでは「次」 ボタンを無効／非表示
 - 新たに検索を実行した場合、ページ番号は1にリセット
 - 検索結果が 0 件のとき
 - 「該当するデータがありません」などのメッセージを表示
 - 一覧部分は空にする
 - 「前」「次」は押せないようにする
-

5.7 エラー処理（一覧画面）

- すべての `fetch` は必ず `try { ... } catch (error) { ... }` で囲む
 - エラー発生時
 - 画面上に「一覧の取得に失敗しました」などのメッセージを表示
 - `console.error` で詳細ログを出力
-

6. 参考 : *_like を使った部分一致サンプル

6.1 概要

JSONPlaceholder は内部的に **json-server** を利用しており、
json-server には *_like という検索用クエリパラメータが存在します。

これを利用すると、以下のように **サーバ側で部分一致検索** を行うこともできます。

6.2 title を部分一致検索するサンプル関数（参考）

※課題のルールに従う場合、この関数を呼び出す側で `try ~ catch` を書いてください。
ここでは *_like の使い方を分かりやすくするため、あえて `try ~ catch` を省いたサンプルです。

```
async function searchPostsWithLike({ userId, title }) {
  const baseUrl = 'https://jsonplaceholder.typicode.com/posts'
  const params = new URLSearchParams()

  // 完全一致フィルタ
  if (userId) {
    params.set('userId', String(userId))
  }

  // サーバ側部分一致フィルタ
  if (title) {
    // title に title の文字列を含むもの
    params.set('title_like', title)
    // 先頭一致にしたい場合は以下のように利用できる場合もある
    // params.set('title_like', `^${title}`)
  }

  const url = `${baseUrl}?${params.toString()}`

  // API実行処理（呼び出し側で try ~ catch を書く想定）
  const response = await fetch(url)
  if (!response.ok) {
    throw new Error('レスポンスエラー')
  }
  const data = await response.json()
  return data
}
```

7. 詳細画面

7.1 役割

- 一覧画面から渡された `id` の投稿詳細を表示する
 - `userId`, `title`, `body` を編集して更新できるようにする
 - 投稿を削除できるようにする
-

7.2 初期表示

1. URL から `id` を取得する
 2. `id` が取得できない／数値に変換できない場合
 - エラーメッセージを表示する
 - 「一覧へ戻る」ための導線（リンクやボタン）を用意する
 3. 正常な `id` の場合
 - `GET /posts/:id` を呼び出して詳細データを取得する
 - 取得したデータをフォームに設定する
 - 投稿 ID (`id`) : 表示のみ
 - ユーザー (`userId`) : 一覧と同じ `select`
 - タイトル (`title`) : `input`
 - 本文 (`body`) : `textarea`
-

7.3 バリデーション（詳細画面・更新時）

- `userId`
 - 必須
 - `title`
 - 必須
 - `body`
 - 必須
 - エラー発生時
 - API は呼び出さず、エラーメッセージを表示する
 - エラーの表示位置やスタイルは実装者に任せる
-

7.4 更新処理（PUT/PATCH）

- 「更新」ボタンを押すと以下の処理を行う
 1. バリデーションを実行する
 2. 問題がなければ `PUT /posts/:id` または `PATCH /posts/:id` を呼び出す
 3. 更新が成功の場合
 - 画面上に「更新に成功しました」などのメッセージを表示する

- `response.json()` の結果を `console.log` で表示する
4. 更新が失敗の場合
 - 画面上に「更新に失敗しました」などのメッセージを表示する
 - `console.error` で詳細を表示する

※JSONPlaceholder の仕様により、更新は **サーバー上には永続化されません**。再度 `GET /posts/:id` を行くと元の値が返ってきます。

7.5 削除処理 (DELETE)

- 「削除」ボタンを押すと以下の処理を行う
1. `DELETE /posts/:id` を呼び出す
 2. 削除が成功の場合
 - 画面上に「削除に成功しました」などのメッセージを表示する
 - レスポンスを `console.log` で表示する
 - 一覧画面に戻る (例: `location.href = 'list.html'`)
 3. 削除が失敗の場合
 - 画面上に「削除に失敗しました」などのメッセージを表示する
 - `console.error` で詳細を表示する

※削除も同様に、JSONPlaceholder 上では永続化されません。一覧を再取得すると該当の投稿が再び表示されます。